

board.py

board.py

PY

game_engine.py

game_engine.py

PY

main.py

main.py

PY

README.md

README.md

MD

The file board.py and game_engine.py is present in in game folder in same repo of main.py

Task 1:

Task 1: Fix the invalid swap move deduction bug

Swapping two gems that do not form any 3-in-a-row combination reverts the gems back to their original tiles, yet the game still docks a remaining move. Ensure moves are only deducted when a swap successfully creates at least one valid match.

Edited a file, looked for files, read a file

Task 1 is fixed in board.py, in process_swap. I haven't run the game, so the fix is untested.

Cause: self.moves_remaining -= 1 ran right after the swap, before the code checked for matches. An invalid swap was then reverted and returned False, but the move had already been deducted.

Fix: The no-match check now comes first, and the deduction only happens after it passes:

python

```
self.swap_gems(pos1, pos2) matches = self.find_matches()
```

```
if not matches: self.swap_gems(pos1, pos2) # revert, no move spent return False
```

```
self.moves_remaining -= 1      # only valid swaps cost a move cleared = self.resolve_matches()
self.score += cleared * 10 return True
```

To verify for your "after" video: run `python main.py`, swap two gems that make no match, and check that "MOVES LEFT" stays the same. Then make a valid swap and check that it drops by 1.

I can move on to Task 2 (cascade combo multiplier) when you're ready.

Task 2: Implement cascade combo multiplier

Gems currently award a flat point value regardless of whether they were matched by the player or cleared via gravity chain reactions. Introduce a cascade combo multiplier that increases score rewards sequentially for every consecutive falling reaction triggered within a single turn.

Edited a file

Task 2 is implemented in `board.py`. I haven't run the game, so it is untested.

How it works: `resolve_matches()` now returns points instead of a gem count. Each pass through its clear-and-refill loop is one cascade step, and the multiplier grows with each step:

Step in the turn Multiplier

Player's own match	×1
1st chain reaction	×2
2nd chain reaction	×3
and so on	×4, ×5, ...

For example, a 3-gem match that triggers one chain of 3 gems scores $3 \times 10 \times 1 + 3 \times 10 \times 2 = 90$, where it used to score 60.

Other changes:

- `process_swap` now does `self.score += self.resolve_matches()`.
- I added a `POINTS_PER_GEM = 10` constant at the top of the file, replacing the hard-coded 10.
- `self.last_combo` records the deepest cascade of the last turn, in case you want to show a "Combo xN!" message in the HUD for the demo video. Say so and I'll add it to `game_engine.py`.
- `reset()` also calls `resolve_matches()` to clear accidental matches on the starting board. That call ignores the score, so the opening board still gives no free points.

To verify: play until a swap triggers a chain. The score should jump by more than the flat 10 per gem would give. Since the starting board is random, a chain can take a few moves to show up.

Task 3 (4-in-a-row line-clear gems) is next whenever you're ready.

Task 3: Implement 4-in-a-row special line-clear gems

Standard match-3 games reward larger combinations. Aligning 4 matching gems in a line should merge them into an enhanced glowing gem that detonates an entire row or column when subsequently matched.

provide the code changes here

Chaining special gem explosions through recursive line clears.

Here are the Task 3 changes. They are all in `board.py`, and `game_engine.py` and `main.py` need no changes. I haven't run any of this.

Design:

- A horizontal run of 4 or more leaves behind a **row-clear** gem, and a vertical run leaves a **column-clear** gem.
- The new gem appears on the cell the player swapped into, or in the middle of the run if it came from a cascade.
- When a special gem is part of any match, its whole row or column is cleared.
- Special gems caught in that line also detonate, so chains work.
- All cleared gems score normally, and the Task 2 combo multiplier still applies.

1. Add `import math` at the top

python

```
import math import random import pygame
```

2. Gem: add a `special` field

python

```
class Gem:
```

```
    def __init__(self, color, target_row, col):        self.color = color        self.special = None
# None, "row" or "col"        self.target_row = target_row        self.col = col        self.current_y =
(target_row - 2) * TILE_SIZE        self.target_y = target_row * TILE_SIZE        self.fall_speed
= 12.0
```

3. Replace `find_matches` with run-based detection

python

```
    def find_runs(self):        """Return a list of (cells, kind) for every run of 3+ same-colour gems.
kind is "row" for horizontal runs and "col" for vertical runs."""        runs = []
```

```

        for r in range(GRID_SIZE):
            c = 0
            while c < GRID_SIZE:
                gem = self.grid[r][c]
                if gem is None:
                    c += 1
                    continue
                end = c + 1
                while (
                    end < GRID_SIZE
                    and self.grid[r][end].color == gem.color
                ):
                    end += 1
                    if end - c >= 3:
                        runs.append([(r, k) for k in range(c, end)], "row")
                        c = end
                        break
            else:
                c = GRID_SIZE

        for c in range(GRID_SIZE):
            r = 0
            while r < GRID_SIZE:
                gem = self.grid[r][c]
                if gem is None:
                    r += 1
                    continue
                end = r + 1
                while (
                    end < GRID_SIZE
                    and self.grid[end][c].color == gem.color
                ):
                    end += 1
                    if end - r >= 3:
                        runs.append([(k, c) for k in range(r, end)], "col")
                        r = end
                        break
            else:
                r = GRID_SIZE

        return runs

    def find_matches(self):
        matched = set()
        for cells, _ in self.find_runs():
            matched.update(cells)
        return matched

```

4. Add three helpers (place them above resolve_matches)

python

```

    def _pick_spawn_cell(self, cells, swapped):
        """Choose which gem of a 4+ run turns into the special gem."""
        candidates = [p for p in swapped if p in cells]
        candidates.append(cells[len(cells) // 2])
        candidates.extend(cells)
        for r, c in candidates:
            if self.grid[r][c] and not self.grid[r][c].special:
                return (r, c)
        return None

    def _expand_specials(self, cells):
        """Add the full row/column of every special gem in `cells`, following chain reactions between special gems."""
        cleared = set(cells)
        queue = [p for p in cleared if self.grid[p[0]][p[1]].special]
        detonated = set()

        while queue:
            r, c = queue.pop()
            if (r, c) in detonated:
                continue
            detonated.add((r, c))
            gem = self.grid[r][c]
            if gem.special == "row":
                line = [(r, k) for k in range(GRID_SIZE)]
            else:
                line = [(k, c) for k in range(GRID_SIZE)]
            for cell in line:
                other = self.grid[cell[0]][cell[1]]
                if other is None:
                    continue
                cleared.add(cell)
                if other.special and cell not in detonated:
                    queue.append(cell)
            return cleared

```

5. Replace resolve_matches

python

```
def resolve_matches(self, swapped=(), spawn_specials=True): """Clear matches until the board is stable and return the points earned.
```

- Each cascade step raises the combo multiplier (Task 2). - Runs of 4+ leave a special gem behind instead of clearing fully. - Special gems in a match detonate their whole row/column.

```
"""
total_points = 0
combo = 0
while True:
    runs = self.find_runs()
    if not runs:
        break
    combo += 1

    matched = set()
    for cells, _ in runs:
        matched.update(cells)

    spawns = {}
    if spawn_specials:
        for cells, kind in runs:
            if len(cells) >= 4:
                cell = self._pick_spawn_cell(
                    cells, swapped if
                    combo == 1 else ()
                )
                if cell:
                    spawns[cell] = kind

    to_clear = self._expand_specials(matched)
    for cell in spawns:
        to_clear.discard(cell)

    total_points += len(to_clear) * POINTS_PER_GEM * combo

    for r, c in to_clear:
        self.grid[r][c] = None
    for (r, c), kind in spawns.items():
        self.grid[r][c].special = kind

    self.drop_and_refill()

    self.last_combo = combo
    return total_points
```

6. reset: don't spawn specials on the starting board

python

```
self.resolve_matches(spawn_specials=False)
```

7. process_swap: pass the swapped cells through

python

```
self.moves_remaining -= 1

self.score += self.resolve_matches(swapped=(pos1, pos2))
return True
```

8. render: draw the glow

Inside the if gem: block, right after the existing white 1px outline is drawn, add:

python

```

        if gem.special:
            pulse = (math.sin(pygame.time.get_ticks() / 150) + 1)
            glow = (255, int(200 + 55 * pulse), int(100 + 155 * pulse))
        pygame.draw.rect(surface, glow, tile_rect, width=4, border_radius=10)
        cx, cy = tile_rect.center
        if gem.special == "row":
            pygame.draw.line(surface, glow, (tile_rect.left + 8, cy), (tile_rect.right - 8, cy), 4)
        else:
            pygame.draw.line(surface, glow, (cx, tile_rect.top + 8), (cx, tile_rect.bottom - 8), 4)

```

The glowing gem shows a pulsing border with a horizontal bar for a row-clear or a vertical bar for a column-clear.

To test for your video:

1. Make a swap that lines up 4 gems. A glowing gem should remain where the swap landed.
2. Match that gem in a later swap. The whole row or column should clear and score.

Four-in-a-row is rare on a random board, so you may need a few attempts. For a quick demo, temporarily lower GEM_COLORS to 3 or 4 colours so matches come up more often.

Limitation: a run that includes a special gem keeps that gem's own effect rather than making a new special. Say if you'd like it handled differently.

Task 4 (the idle hint) is next whenever you're ready.

Task 4: Implement an idle hint indicator

Players can occasionally get stuck scanning for potential combinations. If no input is registered for more than 5 seconds, highlight an available adjacent swap pair with a subtle pulsing or shimmering animation to guide the player forward.

Designing a hint-finding feature for a matching game.

Designing a hint-finding feature for a matching game.

Here are the Task 4 changes. They touch board.py and game_engine.py, and main.py stays the same. I haven't run any of this.

Design:

- Board.find_hint_move() tries every adjacent swap, keeps the first one that would create a match, and always undoes the swap afterwards.
- GameEngine tracks the time of the last input. After more than 5 seconds of idleness, with the board settled and the game not over, it asks the board for a hint.
- The hint pair is drawn with a soft pulsing glow and cleared on the next click or restart.
- It uses math, which you already imported for Task 3.

board.py

1. In `Board.__init__`, add a **hint** attribute next to `self.selected = None`:

python

```
self.selected = None    self.hint = None    # ((r1, c1), (r2, c2)) or None
```

2. Add this method (for example above `process_swap`):

python

```
def find_hint_move(self):    """Return an adjacent pair whose swap creates a match, or
None.    The board is left unchanged. Only call when no gems are animating."""    for r in
range(GRID_SIZE):    for c in range(GRID_SIZE):    for dr, dc in ((0, 1), (1, 0)):
r2, c2 = r + dr, c + dc    if r2 >= GRID_SIZE or c2 >= GRID_SIZE:
continue    self.swap_gems((r, c), (r2, c2))    found =
bool(self.find_matches())    self.swap_gems((r, c), (r2, c2)) # always undo
if found:    return ((r, c), (r2, c2))    return None
```

3. In `reset`, clear the hint:

python

```
self.selected = None    self.hint = None
```

4. At the end of `render`, after the gem-drawing loop and outside it, so it is dedented to the same level as the `for r in range(GRID_SIZE):` line, add:

python

```
if self.hint:    pulse = (math.sin(pygame.time.get_ticks() / 200) + 1) / 2 # 0..1
alpha = int(60 + 140 * pulse)    for r, c in self.hint:    hx = self.offset_x + c *
TILE_SIZE    hy = self.offset_y + r * TILE_SIZE    glow =
pygame.Surface((TILE_SIZE - 4, TILE_SIZE - 4), pygame.SRCALPHA)
pygame.draw.rect(glow, (255, 255, 255, alpha // 2), glow.get_rect(),
border_radius=10,    )    pygame.draw.rect(glow, (255, 255, 120,
alpha), glow.get_rect(), width=4, border_radius=10,    )
surface.blit(glow, (hx + 2, hy + 2))
```

game_engine.py

1. Add a **constant** under the imports:

python

```
HINT_DELAY_MS = 5000
```

2. In `__init__`, after the board is created:

python

```
self.last_input_time = pygame.time.get_ticks()
```

3. Add this as the first line of `handle_click`, before the early return, so any click counts as activity:

python

```
def handle_click(self, mouse_pos):    self.last_input_time = pygame.time.get_ticks()
self.board.hint = None                if self.board.is_game_over() or self.board.is_animating():
return ...
```

4. Update `reset`:

python

```
def reset(self):    self.board.reset()    self.last_input_time = pygame.time.get_ticks()
```

5. Replace `update`:

python

```
def update(self):    self.board.update()

    now = pygame.time.get_ticks()    if self.board.is_animating() or
self.board.is_game_over():        # Idle time only counts while the board is settled and
playable.    self.last_input_time = now    self.board.hint = None    elif
self.board.hint is None and now - self.last_input_time > HINT_DELAY_MS:
self.board.hint = self.board.find_hint_move()
```

How it behaves:

- The 5-second countdown only runs when the board is settled, so a hint never appears mid-cascade.
- Clicking anywhere, including selecting a gem, removes the hint and restarts the timer.
- If the hint is shown and the player stays idle, it keeps pulsing until the next click.
- If no valid move exists, `find_hint_move()` returns `None` and nothing is highlighted. That is a dead board, which the game doesn't handle.
- With Task 3 in place, `find_matches` still works for hint detection, since it just wraps `find_runs`.

To verify for your video: start the game, don't touch anything for about 5 seconds, and two adjacent gems should start pulsing. Swapping those two should produce a match. Click anywhere and the hint should disappear.

That completes all four tasks. For the submission, record the "before" video from the original code (the bug is the move counter dropping on an invalid swap), and the "after" video should show the fixed counter, a cascade score jump, a 4-gem special gem detonating, and the idle hint.

Debugging misplaced code found in a pasted file.

Debugging misplaced code found in a pasted file.

No, `board.py` has four problems, and the first would crash the game on startup. Parts of the `game_engine.py` changes ended up in `board.py`.

1. `reset()` has two engine lines that don't belong there (crash)

Remove `self.board.reset()` and `self.last_input_time = ...` from `Board.reset`. `Board` has no `.board` attribute, so you'd get an `AttributeError`, and it would also recurse. It should be:

python

```
def reset(self):      self.score = 0      self.moves_remaining = self.max_moves
self.selected = None  self.hint = None    for r in range(GRID_SIZE):      ...
```

(`self.last_input_time = ...` belongs in `GameEngine.reset`, as I showed earlier.)

2. `Board.update()` is the engine's update

Restore the original, which just updates the gems:

python

```
def update(self):      for r in range(GRID_SIZE):      for c in range(GRID_SIZE):
if self.grid[r][c]:    self.grid[r][c].update()
```

The idle-timer version (`self.board.update()`, `HINT_DELAY_MS`, and so on) goes in `game_engine.py`'s `update`, not here.

3. The hint-drawing block in `render` is in the wrong place

It's indented inside the `for c` loop, so it runs 64 times per frame. Its `for r, c in self.hint:` also overwrites the loop variables, which breaks the `self.selected == (r, c)` check. Move the whole block out of both loops, dedented to the same level as the first `for r in range(GRID_SIZE):` in `render`, so it runs once after all gems are drawn.

4. Small cleanups (not bugs)

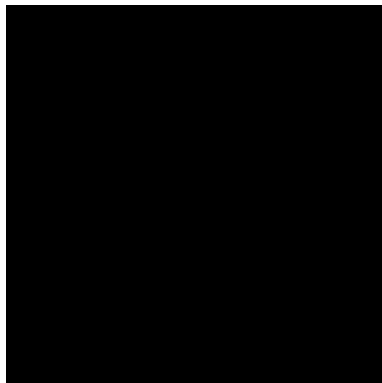
- Remove `self.hint = None` from `Gem.__init__`. The hint belongs to Board.
- Add `self.last_combo = 0` and `self.hint = None` to `Board.__init__` before `self.reset()`.
- `find_runs` and `find_hint_move` are indented 4 spaces too deep. Python accepts it because it's consistent, but it looks odd.
- The old `find_matches` still works. You can optionally replace it with the shorter version I gave, which wraps `find_runs`.

Things that look correct: the Task 1 fix in `process_swap`, the Task 2 multiplier and `POINTS_PER_GEM`, the Task 3 special-gem logic (`_pick_spawn_cell`, `_expand_specials`, `resolve_matches`, and the glow drawing), and `find_hint_move`.

I can't see your `game_engine.py` here. Please check that it has these four things:

- `HINT_DELAY_MS = 5000` at the top.
- `self.last_input_time = pygame.time.get_ticks()` in `__init__` and in `reset`.
- `self.last_input_time = ...` and `self.board.hint = None` as the first lines of `handle_click`.
- The new update with the idle-timer logic.

If you paste that file I'll check it too.



1791377018229_image.png

there is some mistak

Debugging a puzzle board's hint highlighting behavior.

Debugging a puzzle board's hint highlighting behavior.

That screenshot is the render bug I flagged earlier (#3). It's still in your `board.py`.

What's happening: the hint block is indented inside the `for c` loop, and its `for r, c in self.hint:` reuses the names `r` and `c`. After the first cell, `r` is stuck at the hint's row (1). That's

why only column 0 and row 1 get drawn. The other cells are never drawn, so the board looks empty.

Fix: move the hint block out of both loops, so it runs once after all the gems are drawn. The end of render should look like this:

python

```
        for r in range(GRID_SIZE):
            self.grid[r][c] if gem:
            self.offset_y + gem.current_y
            TILE_SIZE - 4)

            pygame.draw.rect(surface, gem.color, tile_rect, border_radius=10)
            pygame.draw.rect(
                surface, (255, 255, 255), tile_rect, width=1,
                border_radius=10
            )
            if gem.special:
                pulse =
                (math.sin(pygame.time.get_ticks() / 150) + 1) / 2
                glow = (255, int(200 + 55 *
                pulse), int(100 + 155 * pulse))
                pygame.draw.rect(surface, glow, tile_rect,
                width=4, border_radius=10)
                cx, cy = tile_rect.center
                if gem.special
                == "row":
                    pygame.draw.line(surface, glow, (tile_rect.left + 8, cy),
                    (tile_rect.right - 8, cy), 4)
                else:
                    pygame.draw.line(surface, glow,
                    (cx, tile_rect.top + 8), (cx, tile_rect.bottom - 8), 4)

            if self.selected == (r, c):
                sel_x = self.offset_x + c * TILE_SIZE
                sel_y = self.offset_y + r * TILE_SIZE
                sel_rect = pygame.Rect(sel_x + 2, sel_y + 2,
                TILE_SIZE - 4, TILE_SIZE - 4)
                pygame.draw.rect(
                    surface, (255,
                    255, 255), sel_rect, width=4, border_radius=10
                )

            # Hint glow: drawn once, after the loops (note the dedent)
            if self.hint:
                pulse =
                (math.sin(pygame.time.get_ticks() / 200) + 1) / 2
                alpha = int(60 + 140 * pulse)
                for
                hr, hc in self.hint:
                    hx = self.offset_x + hc * TILE_SIZE
                    hy = self.offset_y +
                    hr * TILE_SIZE
                    glow = pygame.Surface((TILE_SIZE - 4, TILE_SIZE - 4),
                    pygame.SRCALPHA)
                    pygame.draw.rect(glow, (255, 255, 255, alpha // 2),
                    glow.get_rect(), border_radius=10)
                    pygame.draw.rect(glow, (255, 255, 120, alpha),
                    glow.get_rect(), width=4, border_radius=10)
                    surface.blit(glow, (hx + 2, hy + 2))
```

I also renamed the hint loop variables to `hr`, `hc` so they can't clobber `r` and `c` again. The `if self.hint:` line must line up with the first `for r in range(GRID_SIZE):` in `render`.

While you're in the file, please also apply the other fixes from my last message, in case you haven't yet:

- Remove `self.board.reset()` and `self.last_input_time = ...` from `Board.reset`.

- Restore `Board.update` to just update the gems. The idle-timer version belongs in `game_engine.py`.

After that, the board should fill in normally. The hint should appear after about 5 idle seconds as a pulsing glow on two adjacent gems, and swapping that pair should make a match.

Claude is AI and can make mistakes.

```
turn a list of (cells, kind) for every run of 3+ same-colour gems. kind is "row" for horizontal
runs and "col" for vertical runs.""" runs = [] for r in range(GRID_SIZE): c = 0 while c <
GRID_SIZE: gem = self.grid[r][c] if gem is None: c += 1 continue end = c + 1 while
( end < GRID_SIZE and self.grid[r][end] and self.grid[r][end].color == gem.color ): end +=
1 if end - c >= 3: runs.append([(r, k) for k in range(c, end)], "row") c = end for c in
range(GRID_SIZE): r = 0 while r < GRID_SIZE: gem = self.grid[r][c] if gem is None: r +=
1 continue end = r + 1 while ( end < GRID_SIZE and self.grid[end][c] and
self.grid[end][c].color == gem.color ): end += 1 if end - r >= 3: runs.append([(k, c) for k in
range(r, end)], "col") r = end return runs def find_matches(self): matched = set() for cells,
_ in self.find_runs(): matched.update(cells) return matched
```

4. Add three helpers (place them above `resolve_matches`)

python

```
def _pick_spawn_cell(self, cells, swapped): """Choose which gem of a 4+ run turns into
the special gem.""" candidates = [p for p in swapped if p in cells]
candidates.append(cells[len(cells) // 2]) candidates.extend(cells) for r, c in candidates:
if self.grid[r][c] and not self.grid[r][c].special: return (r, c) return None
```

```
def _expand_specials(self, cells): """Add the full row/column of every special gem in
`cells`, following chain reactions between special gems.""" cleared = set(cells)
queue = [p for p in cleared if self.grid[p[0]][p[1]].special] detonated = set()
```

```
while queue: r, c = queue.pop() if (r, c) in detonated: continue
detonated.add((r, c)) gem = self.grid[r][c] if gem.special == "row": line
= [(r, k) for k in range(GRID_SIZE)] else: line = [(k, c) for k in
range(GRID_SIZE)] for cell in line: other = self.grid[cell[0]][cell[1]]
if other is None: continue cleared.add(cell) if other.special and
cell not in detonated: queue.append(cell) return cleared
```

5. Replace `resolve_matches`

python

```
def resolve_matches(self, swapped=(), spawn_specials=
```